

Cost Model Verification using Flash-attn with Different Configs and Different LX Allocation Policies.

Spyre gives each core 1587 KB of LX scratchpad. When a fused kernel's intermediate buffers do not all fit, the compiler (LX planning) picks which ones stay on chip and which spill to HBM. On the measurements below **that policy choice is worth up to $2.17\times$ of kernel time.**

The cost model is useful for choosing the policy (although the `cpsat` is already doing very good), within limits this note is specific about. It correctly predicts the relative performance among **15 of 15** pairs of coarse-tiled flash-attention variants correctly. It also correctly predicts the relative performance of **95 of 103** and **61 of 65** pairs of shape variants of flash-attn at 32 and 8 cores. And run across the four shipped policies on the same program, it orders **13 of 13** solver pairs correctly — picking out `greedy` and `cpsat` as the fast pair and `firstfit` and `bestfit` as the slow one, on every shape.

1. Terms

LX is a per-core software-managed scratchpad, 1587 KB usable. A buffer that fits is read and written on chip; one that does not is *spilled*, and every access costs HBM bandwidth.

An allocation is the subset of a kernel's intermediates that stay in LX. Only buffers produced *and* consumed inside the kernel are eligible: a graph input must be read from HBM and an output written back, so neither is a choice.

The policies are `layout_solver` settings. All four measurable ones appear below:

policy	how it chooses
<code>greedy</code> (default)	no objective: walks the program in order, keeps each buffer that fits
<code>firstfit</code>	orders by <code>(lifetime - discount) / uses</code> , then places each buffer in the first gap that fits
<code>bestfit</code>	the same ordering, but places each buffer in the tightest gap that fits
<code>cpsat</code>	exact solver maximising retained <code>(reads + 1) x size</code> — a byte objective

`simulated_annealing` also ships and is not measured: it is stochastic, so two runs of one configuration can allocate differently and it is not a reproducible arm.

optimum is not a policy. It is the allocation with the lowest *predicted* time (using the cost model) among all that fit, found by exhaustive search. It is the reference the other rows are scored against.

2. Method

Coarse tiling compiles on the 2026-07-30 build (torch 2.11) and does not on the current one (torch 2.13). That is why the measurements come in two sets, and why the tiled ones are old: they cannot be retaken.

Tiled (section 3, last table): six flash configurations varying the tile counts, on the 2026-07-30 build.

Untiled (everything else): the current build. Head count, sequence length and core count were swept, keeping shapes where one buffer fits the budget but the working set does not — the condition for the allocator to have a choice. Four of the nine that compiled qualified.

How a policy is evaluated. Each policy is run: one compile per (shape, solver) with `LAYOUT_SOLVER` set, and the residency it chose, its features and its time all come from that same compile. The cost model scores those features, so model and device describe the same allocation.

Kernel time moves as the compiler develops, so no number from one build is set beside a number from the other. Each figure is the mean of four rounds, interleaved in alternating order; round-to-round spread is 0.4-1.8 %.

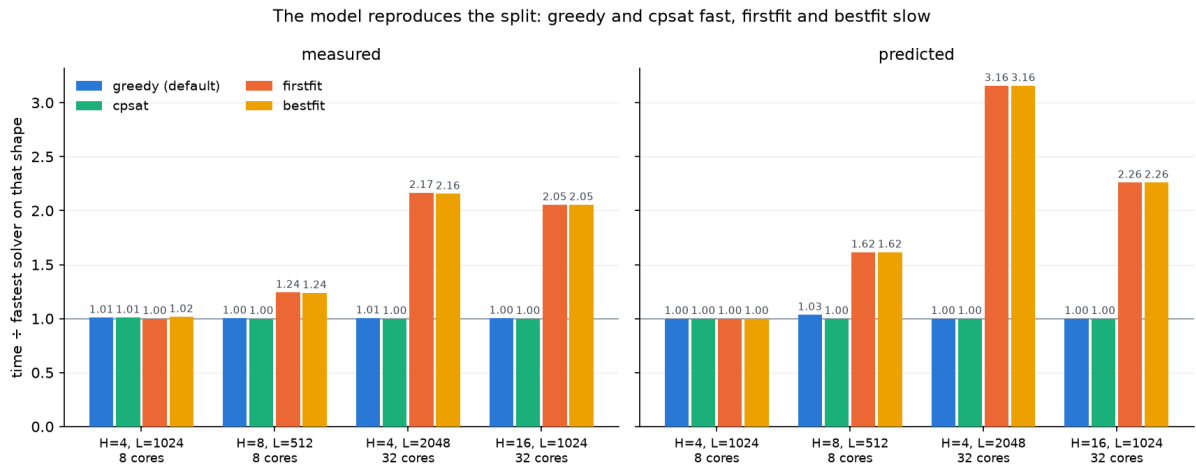
3. Results for different LX allocation policies

One compile per (shape, solver), driven by `LAYOUT_SOLVER`. Each row is what that solver chose, on its own compile — no allocation is reconstructed or pinned.

Each cell: measured microseconds, then in brackets how many buffers that solver placed in LX. One compile per cell.

flash shape	greedy (default)	cpsat	firstfit	bestfit	slowest ÷ fastest
H=4 Lq=Lk=1024, 8 cores	1,586.2 (9)	1,585.3 (9)	1,570.5 (9)	1,597.3 (9)	1.02×
H=4 Lq=Lk=2048, 32 cores	655.9 (13)	652.6 (13)	1,414.0 (12)	1,407.3 (12)	2.17×
H=8 Lq=Lk=512, 8 cores	481.7 (12)	480.3 (12)	596.6 (11)	594.7 (11)	1.24×
H=16 Lq=Lk=1024, 32 cores	777.6 (13)	775.2 (13)	1,592.4 (12)	1,592.4 (12)	2.05×
the model orders 13 of 13 of these pairs correctly					

The cost model, scored on each solver's own features from that solver's own compile:

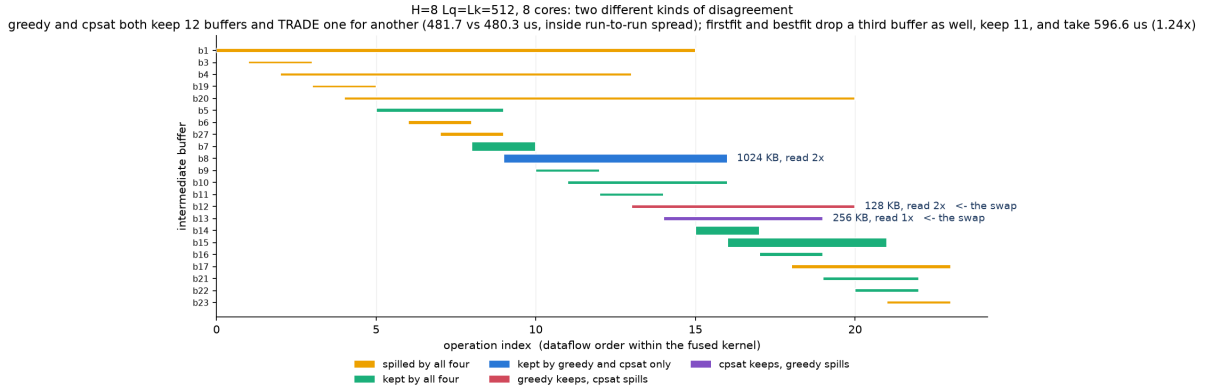


Each solver's time divided by the fastest solver on that shape, measured beside predicted. The two panels have the same shape, which is the result: the model reproduces the split on every configuration, ordering all 13 comparable pairs correctly. It over-states the penalty — 3.16× predicted against 2.17× measured on the widest gap.

Two kinds of disagreement, on one shape. At H=8 Lq=Lk=512 on 8 cores the four policies split three ways. `greedy` and `cpsat` keep the same number of buffers — twelve — and trade one for another:

buffer	per-core	times read	spilling it moves	greedy	cpsat
b12	128 KB	2	384 KB	keeps	spills
b13	256 KB	1	512 KB	spills	keeps

Neither allocation is a subset of the other, so counting resident buffers cannot order this pair. `cpsat`’s choice moves 128 KB less per core and does measure faster, 480.3 against 481.7 us — but 0.3 % is inside the run-to-run spread, so the pair is not evidence either way. `firstfit` and `bestfit` spill `b8` on top of that — 1024 KB per core, read twice — keep eleven, and take 596.6 us. That difference, $1.24\times$, is far outside noise. Every movable buffer, drawn across the operations it is live for, with height proportional to per-core size:



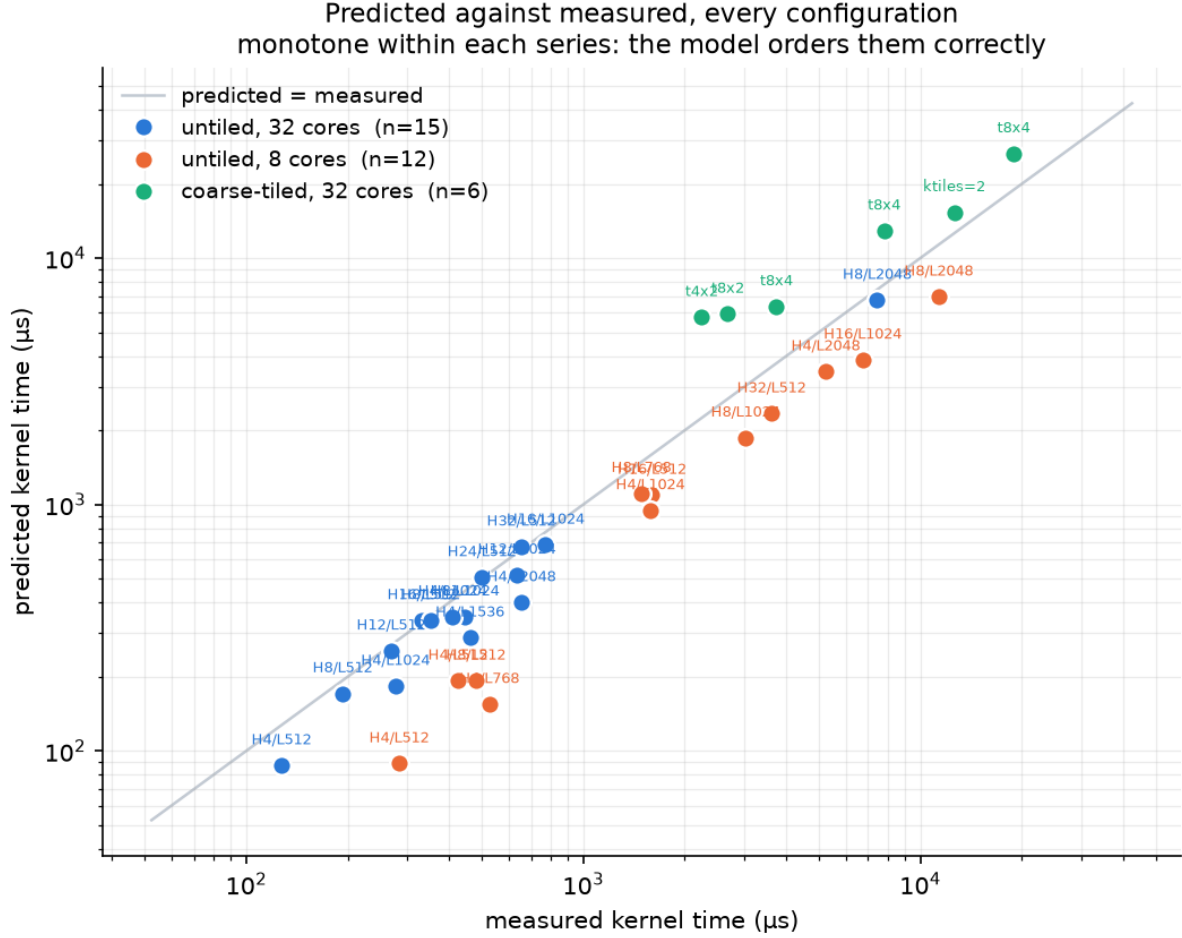
H=8 Lq=Lk=512 on 8 cores. Colour is which policies keep the buffer. Red and purple are the trade: `greedy` keeps `b12` and `cpsat` keeps `b13` instead, same count on both sides. Blue is `b8`, which `firstfit` and `bestfit` spill as well — the difference that is large enough to measure.

The model ranks the policies correctly — 13 of 13 comparable pairs. It separates the fast pair from the slow pair on every shape, and predicts the size of the gap: 400 against 1264 us where the device measures 656 against 1414.

Its absolute predictions run low, $0.39\times$ to $0.97\times$. The ordering is what a scheduler needs and what this note reports; the scale error is a separate matter, tracked in `cost_model_report.md`.

4. Results for different configurations

A different question from sections 3 and 4: comparing whole shapes and tilings against each other, rather than allocations of one program. Across 27 distinct untiled shapes — eight measured twice, agreeing to 0.0-1.0 %, counted once at the mean:



Predicted against measured for every configuration measured, on log axes. Each series is monotone — as measured time rises, so does predicted — which is what ordering correctly looks like. The 8-core points sit on their own band below the 32-core ones: a scale offset, not a ranking error.

5. Discussion

No case was found where `cpsat` is not the best choice. It is fastest or tied on three of the four shapes; on the fourth all four solvers chose the identical allocation, so the 0.9 % it trails by is run-to-run noise.

What it does establish is that the model can be used for early feedback in LX planning. Given each policy's own compile, it orders all 13 comparable pairs correctly — separating `greedy` and `cpsat` from `firstfit` and `bestfit` on every shape, and estimating the gap at the right scale. It orders whole configurations correctly as well, across 27 untiled shapes and 6 coarse-tiled ones. That is the property a planning pass needs: an allocation can be scored before it is compiled and run, and the ranking holds even though the absolute times do not.